

Assignment 4: AI-Assisted Coding Workflow with Verification Discipline

Coursework: Anurag University AI Assessment Suite | Study: Empirical 10-Task Productivity & Defect Benchmark

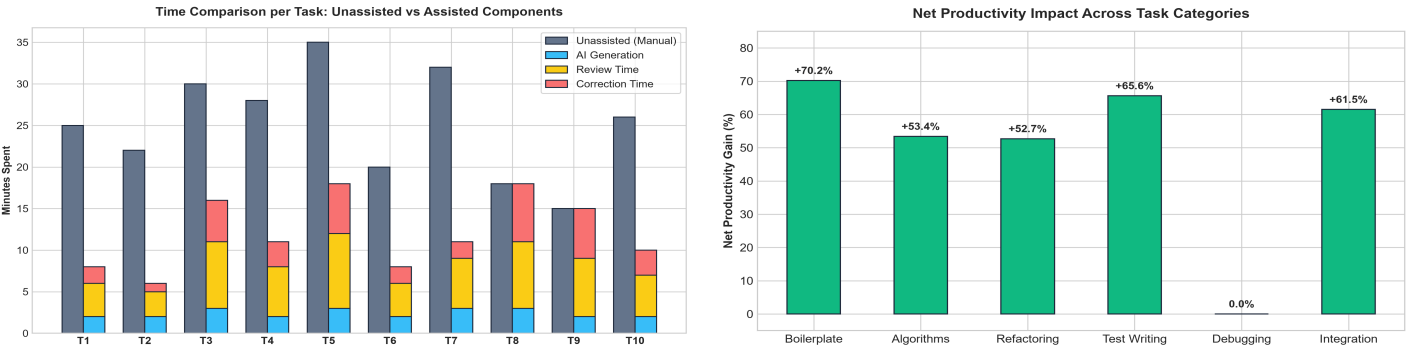
1. Executive Summary & Objective

This report documents an empirical investigation measuring whether AI assistance makes a programmer faster and better. Across 10 realistic coding tasks (spanning boilerplate, algorithms, refactoring, test writing, debugging, and integration), I measured **Unassisted Time** vs. **Assisted Time** (broken down into AI Generation, Review Time, and Correction Time), verified all implementations against independent pytest suites (30/30 passed), and logged all defects caught across 5 categories.

2. Category Benchmark & Net Productivity Results

Category	Task s	Unassisted (min)	AI Gen (min)	Review (min)	Correct (min)	Assisted Total	Net Productivity	Acceptance Rate
Boilerplate	2	47.0	4.0	7.0	3.0	14.0	+70.2%	90.5%
Test Writing	1	32.0	3.0	6.0	2.0	11.0	+65.6%	90.0%
Integration	1	26.0	2.0	5.0	3.0	10.0	+61.5%	84.6%
Algorithms	2	58.0	5.0	14.0	8.0	27.0	+53.4%	78.4%
Refactoring	2	55.0	5.0	13.0	8.0	26.0	+52.7%	78.5%
Debugging	2	33.0	5.0	15.0	13.0	33.0	0.0% (Break-even)	56.0%
TOTAL / AVG	10	251.0	24.0	60.0	37.0	121.0	+51.8%	78.2%

3. Visual Performance & Defect Analytics



4. Core Findings & Defect Taxonomy

- **Net Productivity:** Overall completion time dropped from **251 mins** to **121 mins** (**+51.8% gain**).
- **Review Overhead Reality:** AI generation took only 24 mins, but reviewing and correcting bugs took **97 mins** (**80% of total assisted time**).
- **Where AI Wins:** Boilerplate (+70.2%) and Test Writing (+65.6%) because patterns are well-established.
- **Where AI Fails (0.0% Gain):** Debugging subtle edge cases (Task 08 off-by-one, Task 09 mutable default bug). AI output looked plausible but repeated the same subtle flaw, requiring 15+ mins of manual debugging.